A01 - MongoDB - Modelling and CRUD in PHP

#modern_databases #mongodb #php

- ***Name:*** Zoe McFife
- ***Legal Name and Student NR:*** Bunea, S2510238021
- ***Time Spent***: 05:25:31

---

1 Model the Island

1.1 Requirements

> An island has a **name**, the player who **owns** it, a **hemisphere** (north or south), a **creation date** and, if known, a **native fruit**. **Residents**: at most **ten villagers** of the catalogue live on an island; the island page always shows their **names**. **Inventory**: **items** of the **catalogue** with a **count**, a few **hundred entries at most**, changed **many times a day**. **Visits**: every time another player visits, the game records **who came and when**, and keeps that forever.

1.2 JSON Example

Islands

```
[
    {
        "_id": ObjectId("6ad3622d82f911e9df683b1d"),
        "islandName": 'Zoe\'s Bizarre Island',
        "owner": 'Zoe McFife',
        "hemisphere": 'north',
        "creation_date": ISODate('2024-10-17T10:00:00Z'),
        "native_fruit": 'Strawberry',
        "residents":
        [
            {
                "villagerName": 'Ankha',
                "villagerId": 13,
            },
            {
                "villagerName": 'Audie',
                "villagerId": 20,
            },
        ],
        "inventory":
        [
            {
                "itemId": 4483,
                "itemName": 'cat nose',
                "count": 242
            },
            {
                "itemId": 4672,
                "itemName": 'curly mustache',
                "count": 1
            }
        ]
    }
]
```

Visits

```
[
    {
        _id: ObjectId('6ad3631a5be09c155729ba16'),
        islandId: ObjectId("6ad3622d82f911e9df683b1d"),
        islandName: 'Zoe\'s Bizarre Island',
        player: 'Kirsikka Lumikettu',
        visit_time: ISODate('2025-11-11T05:25:21Z')
    },
    {
        _id: ObjectId('6ad3631dbf4803c16c56b1da'),
        islandId: ObjectId("6ad3622d82f911e9df683b1d"),
        islandName: 'Zoe\'s Bizarre Island',
        player: 'Blerg',
        visit_time: ISODate('2025-01-11T15:05:21Z')
    },
    {
        _id: ObjectId('6ad36320547a1b52da9227d5'),
        islandId: ObjectId("6ad3622d82f911e9df683b1d"),
        islandName: 'Zoe\'s Bizarre Island',
        player: 'Mina Airfryer',
        visit_time: ISODate('2026-10-17T03:00:21Z')
    },
]
```

1.3 Explanation

Residents - Embedded

The island doesn’t need to know more about the residents than their name (copied) and ID. When resident information is needed, should be fetched from the villagers collection. The

- **Read with parent?** — Yes (only name needed)
- **How many / bounded?** — Few, bounded
- **Shared?** — Yes
- **Changes** — On its own

Inventories - Embedded

Inventories belong to the island. Doesn’t grow that large. Items in the inventory referenced through itemIds. Item names copied.

- **Read with parent?** — Yes
- **How many / bounded?** — A bunch but bounded
- **Shared?** — Not really
- **Changes** — many times a day

Visits

Visits should be their own collection. They reference which island they belong to and keep a copy of the name for easier querying.

- **Read with parent?** — No
- **How many / bounded?** — Many, not bounded
- **Shared?** — No
- **Changes** — On its own

2 The Data Access Layer

2.1 IslandDAO

Implemented required DAO methods. Collection gets injected in the constructor.

```php
<?php

require_once __DIR__ . '/../connect.php';

use MongoDB\BSON\ObjectId;
use MongoDB\Collection;
use MongoDB\BSON\UTCDateTime;

class IslandDao
{
    private const MAX_RESIDENTS = 10;

    public function __construct(private readonly Collection $islands) {}

    public function create(string $name, string $owner, string $hemisphere, ?string $nativeFruit = null) :
ObjectId
    {
        if (!in_array($hemisphere, ['north', 'south'], true))
        {
            throw new InvalidArgumentException('hemisphere must be north or south');
        }

        $island =
            [
                'islandName' => $name,
                'owner' => $owner,
                'hemisphere' => $hemisphere,
                'creationDate' => new UTCDateTime(),
                'residents' => [],
                'inventory' => [],
            ];

    if ($nativeFruit !== null)
    {
        $island['nativeFruit'] = $nativeFruit;
    }

    return $this->islands->insertOne($island)->getInsertedId();
}

public function findById(ObjectId $id): ?object
{
    return $this->islands->findOne(['_id' => $id]);
}

public function findByName(string $name) : ?object
{
    return $this->islands->findOne(['islandName' => $name]);
}

public function findByResident(string $residentName) : Array
{
    return $this->islands->find(['residents' => $residentName])->toArray();
}

public function moveIn(string $name, int $villagerId, string $villagerName) : bool
{
    $result = $this->islands->updateOne
    (
        [
            'islandName' => $name,
            'residents' => ['$ne' => $villagerId],
            'residents.' . (self::MAX_RESIDENTS - 1) => ['$exists' => false]
        ],
        [
            '$push' => ['residents' => ['villagerName' => $villagerName, 'villagerId' => $villagerId]],
        ]
    );

    return $result->getModifiedCount() === 1;
}
```

```
public function moveOut(string $name, int $villagerId) : bool
{
    $result = $this->islands->updateOne
    (
        [
            'islandName' => $name
        ],
        [
            '$pull' => ['residents' => ['villagerId' => $villagerId]],
        ]
    );

    return $result->getModifiedCount() === 1;
}

public function addToInventory(string $name, int $itemId, string $itemName, int $count) : void
{
    if ($count === 0)
    {
        return;
    }

    $updated = $this->islands->updateOne
    (
        [
            'islandName' => $name,
            'inventory.itemId' => $itemId,
        ],
        [
            '$inc' => ['inventory.$.count' => $count],
        ]
    );

    if ($updated->getModifiedCount() === 0)
    {
        $this->islands->updateOne
        (
            [
                'islandName' => $name
            ],
            [
                '$push' => ['inventory' => ['itemId' => $itemId, 'itemName' => $itemName, 'count' => $count]],
            ]
        );
    }
}

public function rename(string $old, string $new) : bool
{
    if ($new === $old)
        return false;

    $result = $this->islands->updateOne
    (
        [
            'islandName' => $old
        ],
        [
            '$rename' => ['name' => $new]
        ]
    );

    return $result->getModifiedCount() === 1;
}

public function delete(string $name) : bool
{
    $result = $this->islands->deleteOne
    (
        [
```

```
                'islandName' => $name
            ]
        );

        return $result->getDeletedCount() === 1;
    }
}
```

2.2 VisitDAO

`VisitDAO` implementing `record` and `latest`.

```
<?php

use MongoDB\BSON\ObjectId;
use MongoDB\BSON\UTCDateTime;
use MongoDB\Collection;

class VisitDao
{
    public function __construct(private readonly Collection $visits) {}

    public function record(ObjectId $islandId, string $islandName, string $visitor, UTCDateTime $visitTime) :
ObjectId
    {
        $visit =
            [
                'visitor' => $visitor,
                'islandName' => $islandName,
                'islandId' => $islandId,
                'visitTime' => $visitTime,
            ];

        return $this->visits->insertOne($visit)->getInsertedId();
    }

    public function latest(ObjectId $islandId, int $n) : array
    {
        if ($n <= 0)
            return [];

        return $this->visits->find
        (
            [
                'islandId' => $islandId
            ],
            [
                'sort' => ['visitTime' => -1],
                'limit' => $n
            ]
        )->toArray();
    }
}
```

2.3 Demo

2.3.1 Code

```php
<?php

use MongoDB\BSON\UTCDateTime;

require_once __DIR__ . '/../connect.php';
require_once 'IslandDao.php';
require_once 'VisitDao.php';

$islands = $mongo->catalogue->islands;
$visits = $mongo->catalogue->visits;

$islandDAO = new IslandDao($islands);
$visitDAO = new VisitDao($visits);

// Creating three islands

$island1 = $islandDAO->create("Zoe's Bizarre Island", "Zoe McFife", "north", "Strawberry");
$island2 = $islandDAO->create("Blerg Islands", "Blerg", "south");
$island3 = $islandDAO->create("Snow", "Kirsikka Lumikettu", "north", "Pancake");

echo "Islands: \n";

var_dump($islandDAO->findById($island1));
var_dump($islandDAO->findById($island2));
var_dump($islandDAO->findById($island3));

echo "\n";

// adding villagers

echo "Adding Villagers: \n\n";

echo $islandDAO->moveIn("Zoe's Bizarre Island", 13, "Ankha") . "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 20, "Audie"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 1, "Admiral"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 3, "Agnes"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 17, "Apollo"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 33, "Bella"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 48, "Bob"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 89, "Cleo"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 93, "Coco"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 106, "Daisy"). "\n";
echo $islandDAO->moveIn("Zoe's Bizarre Island", 112, "Diana"). "\n";

echo "expecting - false two times: \n";
echo ($islandDAO->moveIn("Zoe's Bizarre Island", 112, "Diana")). "\n"; // duplicate
echo ($islandDAO->moveIn("Zoe's Bizarre Island", 36, "Bertha")). "\n"; // sorry Bertha, we're full


// removing villagers

echo "Removing villager: \n";
echo $islandDAO->moveOut("Zoe's Bizarre Island", 48). "\n";

echo "all villagers: \n\n";

var_dump(($islandDAO->findByName("Zoe's Bizarre Island")->residents));

echo "\n";

// adding items

$islandDAO->addToInventory("Zoe's Bizarre Island", 4483, "cat nose", 24);
$islandDAO->addToInventory("Zoe's Bizarre Island", 4483, "cat nose", 24);
$islandDAO->addToInventory("Zoe's Bizarre Island", 4672, "curly moustache", 1);

echo "Inventory: cat nose -> 48 ; moustache -> 1\n\n";
```

```php
var_dump(($islandDAO->findByName("Zoe's Bizarre Island")->inventory));

echo "\n";

// visits

// let a clanker make the array
$visits = [
    ["Kirsikka Lumikettu",        '2026-08-18 09:12'],
    ["Blerg",                     '2026-08-21 17:45'],
    ["Mina Airfryer",             '2026-08-24 20:03'],
    ["Jade Arli",                 '2026-08-27 11:30'],
    ["Florence Meean'kar",        '2026-08-30 15:18'],
    ["Jack Storm",                '2026-09-02 08:55'],
    ["William Storm",             '2026-09-05 19:40'],
    ["Captain Nathaniel Storm",   '2026-09-08 13:22'],
    ["Sasha Moi",                 '2026-09-11 21:07'],
    ["Chris Roberts",             '2026-09-14 10:15'],
    ["Kirsikka Lumikettu",        '2026-09-17 16:48'],
    ["Jack Storm",                '2026-09-20 12:01'],
    ["Blerg",                     '2026-09-23 18:36'],
    ["Jade Arli",                 '2026-09-26 09:59'],
    ["Mina Airfryer",             '2026-09-29 14:27'],
    ["Florence Meean'kar",        '2026-10-02 20:50'],
    ["Sasha Moi",                 '2026-10-06 11:11'],
    ["Chris Roberts",             '2026-10-10 17:33'],
    ["Captain Nathaniel Storm",   '2026-10-14 08:42'],
    ["William Storm",             '2026-10-17 03:00'],
];

echo "Visits: \n \n";

foreach ($visits as [$player, $time])
{
    $at = new UTCDateTime(new DateTimeImmutable($time, new DateTimeZone('UTC')));
    echo $visitDAO->record($island1, $player, $at). "\n";
}

echo "\n";

echo "visit vardump: \n\n";

var_dump($visitDAO->latest($island1, 20));
```

2.3.2 Output

> **Note**
>
> after `expecting false two times`, two empty rows got outputted. Expected behavior.

```
PS R:\GitHub\MDB3-STACK\app> docker compose exec php php a01/demo.php
Islands:
object(MongoDB\Model\BSONDocument)#49 (1) {
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#37 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["islandName"]=>
    string(20) "Zoe's Bizarre Island"
    ["owner"]=>
    string(10) "Zoe McFife"
    ["hemisphere"]=>
    string(5) "north"
    ["creationDate"]=>
    object(MongoDB\BSON\UTCDateTime)#38 (1) {
      ["milliseconds"]=>
      string(13) "1792245218277"
    }
    ["residents"]=>
    object(MongoDB\Model\BSONArray)#47 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
    ["inventory"]=>
    object(MongoDB\Model\BSONArray)#48 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
    ["nativeFruit"]=>
    string(10) "Strawberry"
  }
}
object(MongoDB\Model\BSONDocument)#45 (1) {
  ["storage":"ArrayObject":private]=>
  array(7) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#46 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f3"
    }
    ["islandName"]=>
    string(13) "Blerg Islands"
    ["owner"]=>
    string(5) "Blerg"
    ["hemisphere"]=>
    string(5) "south"
    ["creationDate"]=>
    object(MongoDB\BSON\UTCDateTime)#42 (1) {
      ["milliseconds"]=>
      string(13) "1792245218315"
    }
    ["residents"]=>
    object(MongoDB\Model\BSONArray)#41 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
    ["inventory"]=>
    object(MongoDB\Model\BSONArray)#44 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
  }
}
object(MongoDB\Model\BSONDocument)#36 (1) {
```

```
  ["storage":"ArrayObject":private]=>
  array(8) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#48 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f4"
    }
    ["islandName"]=>
    string(4) "Snow"
    ["owner"]=>
    string(18) "Kirsikka Lumikettu"
    ["hemisphere"]=>
    string(5) "north"
    ["creationDate"]=>
    object(MongoDB\BSON\UTCDateTime)#37 (1) {
      ["milliseconds"]=>
      string(13) "1792245218317"
    }
    ["residents"]=>
    object(MongoDB\Model\BSONArray)#38 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
    ["inventory"]=>
    object(MongoDB\Model\BSONArray)#47 (1) {
      ["storage":"ArrayObject":private]=>
      array(0) {
      }
    }
    ["nativeFruit"]=>
    string(7) "Pancake"
  }
}

Adding Villagers:

1
1
1
1
1
1
1
1
1
1

expecting - false two times:


Removing villager:
1
all villagers:

object(MongoDB\Model\BSONArray)#56 (1) {
  ["storage":"ArrayObject":private]=>
  array(9) {
    [0]=>
    object(MongoDB\Model\BSONDocument)#42 (1) {
      ["storage":"ArrayObject":private]=>
      array(2) {
        ["villagerName"]=>
        string(5) "Ankha"
        ["villagerId"]=>
        int(13)
      }
    }
    [1]=>
    object(MongoDB\Model\BSONDocument)#41 (1) {
      ["storage":"ArrayObject":private]=>
      array(2) {
```

```
      ["villagerName"]=>
      string(5) "Audie"
      ["villagerId"]=>
      int(20)
    }
  }
  [2]=>
  object(MongoDB\Model\BSONDocument)#49 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(7) "Admiral"
      ["villagerId"]=>
      int(1)
    }
  }
  [3]=>
  object(MongoDB\Model\BSONDocument)#50 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(5) "Agnes"
      ["villagerId"]=>
      int(3)
    }
  }
  [4]=>
  object(MongoDB\Model\BSONDocument)#51 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(6) "Apollo"
      ["villagerId"]=>
      int(17)
    }
  }
  [5]=>
  object(MongoDB\Model\BSONDocument)#52 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(5) "Bella"
      ["villagerId"]=>
      int(33)
    }
  }
  [6]=>
  object(MongoDB\Model\BSONDocument)#53 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(4) "Cleo"
      ["villagerId"]=>
      int(89)
    }
  }
  [7]=>
  object(MongoDB\Model\BSONDocument)#54 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(4) "Coco"
      ["villagerId"]=>
      int(93)
    }
  }
  [8]=>
  object(MongoDB\Model\BSONDocument)#55 (1) {
    ["storage":"ArrayObject":private]=>
    array(2) {
      ["villagerName"]=>
      string(5) "Daisy"
```

```
        ["villagerId"]=>
        int(106)
      }
    }
  }
}

Inventory: cat nose → 48 ; moustache → 1

object(MongoDB\Model\BSONArray)#59 (1) {
  ["storage":"ArrayObject":private]=>
  array(2) {
    [0]=>
    object(MongoDB\Model\BSONDocument)#47 (1) {
      ["storage":"ArrayObject":private]=>
      array(3) {
        ["itemId"]=>
        int(4483)
        ["itemName"]=>
        string(8) "cat nose"
        ["count"]=>
        int(48)
      }
    }
    [1]=>
    object(MongoDB\Model\BSONDocument)#48 (1) {
      ["storage":"ArrayObject":private]=>
      array(3) {
        ["itemId"]=>
        int(4672)
        ["itemName"]=>
        string(15) "curly moustache"
        ["count"]=>
        int(1)
      }
    }
  }
}

Visits:

6ad37de25fc5710b9b0424f5
6ad37de25fc5710b9b0424f6
6ad37de25fc5710b9b0424f7
6ad37de25fc5710b9b0424f8
6ad37de25fc5710b9b0424f9
6ad37de25fc5710b9b0424fa
6ad37de25fc5710b9b0424fb
6ad37de25fc5710b9b0424fc
6ad37de25fc5710b9b0424fd
6ad37de25fc5710b9b0424fe
6ad37de25fc5710b9b0424ff
6ad37de25fc5710b9b042500
6ad37de25fc5710b9b042501
6ad37de25fc5710b9b042502
6ad37de25fc5710b9b042503
6ad37de25fc5710b9b042504
6ad37de25fc5710b9b042505
6ad37de25fc5710b9b042506
6ad37de25fc5710b9b042507
6ad37de25fc5710b9b042508

visit vardump:

array(20) {
  [0]=>
  object(MongoDB\Model\BSONDocument)#38 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#47 (1) {
        ["oid"]=>
```

```
        string(24) "6ad37de25fc5710b9b042508"
      }
      ["visitor"]=>
      string(13) "William Storm"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#46 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#36 (1) {
        ["milliseconds"]=>
        string(13) "1792206000000"
      }
    }
  }
  [1]=>
  object(MongoDB\Model\BSONDocument)#55 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#44 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042507"
      }
      ["visitor"]=>
      string(23) "Captain Nathaniel Storm"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#45 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#57 (1) {
        ["milliseconds"]=>
        string(13) "1791967320000"
      }
    }
  }
  [2]=>
  object(MongoDB\Model\BSONDocument)#53 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#54 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042506"
      }
      ["visitor"]=>
      string(13) "Chris Roberts"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#58 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#42 (1) {
        ["milliseconds"]=>
        string(13) "1791653580000"
      }
    }
  }
  [3]=>
  object(MongoDB\Model\BSONDocument)#50 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#51 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042505"
      }
      ["visitor"]=>
```

```
      string(9) "Sasha Moi"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#49 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#41 (1) {
        ["milliseconds"]=>
        string(13) "1791285060000"
      }
    }
  }
  [4]=>
  object(MongoDB\Model\BSONDocument)#62 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#52 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042504"
      }
      ["visitor"]=>
      string(18) "Florence Meean'kar"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#56 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#61 (1) {
        ["milliseconds"]=>
        string(13) "1790974200000"
      }
    }
  }
  [5]=>
  object(MongoDB\Model\BSONDocument)#66 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#63 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042503"
      }
      ["visitor"]=>
      string(13) "Mina Airfryer"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#64 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#65 (1) {
        ["milliseconds"]=>
        string(13) "1790692020000"
      }
    }
  }
  [6]=>
  object(MongoDB\Model\BSONDocument)#70 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#67 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b042502"
      }
      ["visitor"]=>
      string(9) "Jade Arli"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#68 (1) {
```

```
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#69 (1) {
      ["milliseconds"]=>
      string(13) "1790416740000"
    }
  }
}
[7]=>
object(MongoDB\Model\BSONDocument)#74 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#71 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b042501"
    }
    ["visitor"]=>
    string(5) "Blerg"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#72 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#73 (1) {
      ["milliseconds"]=>
      string(13) "1790188560000"
    }
  }
}
[8]=>
object(MongoDB\Model\BSONDocument)#78 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#75 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b042500"
    }
    ["visitor"]=>
    string(10) "Jack Storm"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#76 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#77 (1) {
      ["milliseconds"]=>
      string(13) "1789905660000"
    }
  }
}
[9]=>
object(MongoDB\Model\BSONDocument)#82 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#79 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424ff"
    }
    ["visitor"]=>
    string(18) "Kirsikka Lumikettu"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#80 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
```

```
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#81 (1) {
      ["milliseconds"]=>
      string(13) "1789663680000"
    }
  }
}
[10]=>
object(MongoDB\Model\BSONDocument)#86 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#83 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424fe"
    }
    ["visitor"]=>
    string(13) "Chris Roberts"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#84 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#85 (1) {
      ["milliseconds"]=>
      string(13) "1789380900000"
    }
  }
}
[11]=>
object(MongoDB\Model\BSONDocument)#90 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#87 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424fd"
    }
    ["visitor"]=>
    string(9) "Sasha Moi"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#88 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#89 (1) {
      ["milliseconds"]=>
      string(13) "1789160820000"
    }
  }
}
[12]=>
object(MongoDB\Model\BSONDocument)#94 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#91 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424fc"
    }
    ["visitor"]=>
    string(23) "Captain Nathaniel Storm"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#92 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#93 (1) {
      ["milliseconds"]=>
```

```
        string(13) "1788873720000"
      }
    }
  }
  [13]=>
  object(MongoDB\Model\BSONDocument)#98 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#95 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424fb"
      }
      ["visitor"]=>
      string(13) "William Storm"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#96 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#97 (1) {
        ["milliseconds"]=>
        string(13) "1788637200000"
      }
    }
  }
  [14]=>
  object(MongoDB\Model\BSONDocument)#102 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#99 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424fa"
      }
      ["visitor"]=>
      string(10) "Jack Storm"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#100 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#101 (1) {
        ["milliseconds"]=>
        string(13) "1788339300000"
      }
    }
  }
  [15]=>
  object(MongoDB\Model\BSONDocument)#106 (1) {
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#103 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f9"
      }
      ["visitor"]=>
      string(18) "Florence Meean'kar"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#104 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#105 (1) {
        ["milliseconds"]=>
        string(13) "1788103080000"
      }
    }
  }
```

```
}
[16]=>
object(MongoDB\Model\BSONDocument)#110 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#107 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f8"
    }
    ["visitor"]=>
    string(9) "Jade Arli"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#108 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#109 (1) {
      ["milliseconds"]=>
      string(13) "1787830200000"
    }
  }
[17]=>
object(MongoDB\Model\BSONDocument)#114 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#111 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f7"
    }
    ["visitor"]=>
    string(13) "Mina Airfryer"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#112 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#113 (1) {
      ["milliseconds"]=>
      string(13) "1787601780000"
    }
  }
}
[18]=>
object(MongoDB\Model\BSONDocument)#118 (1) {
  ["storage":"ArrayObject":private]=>
  array(4) {
    ["_id"]=>
    object(MongoDB\BSON\ObjectId)#115 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f6"
    }
    ["visitor"]=>
    string(5) "Blerg"
    ["islandId"]=>
    object(MongoDB\BSON\ObjectId)#116 (1) {
      ["oid"]=>
      string(24) "6ad37de25fc5710b9b0424f2"
    }
    ["visitTime"]=>
    object(MongoDB\BSON\UTCDateTime)#117 (1) {
      ["milliseconds"]=>
      string(13) "1787334300000"
    }
  }
}
[19]=>
object(MongoDB\Model\BSONDocument)#122 (1) {
```

```
    ["storage":"ArrayObject":private]=>
    array(4) {
      ["_id"]=>
      object(MongoDB\BSON\ObjectId)#119 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f5"
      }
      ["visitor"]=>
      string(18) "Kirsikka Lumikettu"
      ["islandId"]=>
      object(MongoDB\BSON\ObjectId)#120 (1) {
        ["oid"]=>
        string(24) "6ad37de25fc5710b9b0424f2"
      }
      ["visitTime"]=>
      object(MongoDB\BSON\UTCDateTime)#121 (1) {
        ["milliseconds"]=>
        string(13) "1787044320000"
      }
    }
  }
}
PS R:\GitHub\MDB3-STACK\app>
```

3 Queries

For the SQL queries, I will be assuming a normalized PostgreSQL database.

3.1 Query 1

MongoDB Query

```php
// Query 1 - All islands where villager 1 (Admiral) lives, names only

echo "Query 1: \n";

$islandsWithAdmiral =
    $islands->find
    (
        [
            'residents.villagerName' => 'Admiral'
        ],
        [
            'projection' => ['islandName' => 1]
        ]
    );

foreach ($islandsWithAdmiral as $island)
{
    echo $island->islandName . "\n";
}

echo "\n";
```

Result

```
Query 1:
Misty Cay
Cozy Shores
Lazy Key
Misty Haven
Tiny Shores
Lazy Haven
```

SQL Equivalent

```sql
SELECT islandName FROM islands
NATURAL JOIN islands_villagers
NATURAL JOIN villagers
WHERE villagers.name = "Admiral";
```

3.2 Query 2

MongoDB Query

```php
// Query 2 - All islands whose inventory has at least three clay furnaces (item 717)

echo "Query 2: \n";

$islandsWithThreeClayFurnaces =
    $islands->find(
        [
            'inventory' => [
                '$elemMatch' => [
                    'itemId' => 717,
                    'count'  => ['$gte' => 3],
                ],
            ],
        ],
        [
            'projection' => ['islandName' => 1, 'inventory' => 1],
        ]
    );

foreach ($islandsWithThreeClayFurnaces as $island)
{
    echo $island->islandName . "\n";
    foreach ($island->inventory as $inventory)
    {
        if ($inventory->itemId == 717)
        {
            echo $inventory->itemName . "\n";
            echo $inventory->count . "\n";
        }


    }
    echo "\n";
}

echo "\n";
```

Result

```
Query 2:
Wild Shores
clay furnace
12

Wild Atoll
clay furnace
20

Lost Cay
clay furnace
4

Lazy Key
clay furnace
7

Cozy Atoll
clay furnace
3
```

SQL Equivalent

```sql
SELECT islandName FROM islands
NATURAL JOIN inventory_islands
NATURAL JOIN items
WHERE itemName = "clay furnace"
GROUP BY islandId
HAVING count(items) ⩾ 3;
```

3.3 Query 3

MongoDB Query

```php
// Query 3 - the five newest visits to one island in October 2026, newest first

echo "Query 3: \n";

$islandEx3 = $islands->findOne
(
    [
        'islandName' => "Snowy Shores"
    ]
);

$utc = new DateTimeZone('UTC');

$fiveNewest =
    $visits->find
    (
        [
            'islandId' => $islandEx3->_id,
            'visitTime' => [
                '$gte' => new UTCDateTime(new DateTimeImmutable('2026-10-01 00:00:00', $utc)),
                '$lt'  => new UTCDateTime(new DateTimeImmutable('2026-11-01 00:00:00', $utc)),
            ]
        ],
        [
            'sort' => ['visitTime' => -1],
            'limit' => 5
        ]
    );

foreach ($fiveNewest as $visit)
{
    echo $visit->visitor . "\n";
    echo $visit->visitTime . "\n";
    echo "\n";
}

echo "\n";
```

Result

```
Query 3:
Kirsikka Lumikettu
1792142240000

Chris Roberts
1792139665000

Mina Airfryer
1791744760000

Jade Arli
1791728714000

Kirsikka Lumikettu
1791639243000
```

SQL Equivalent

```sql
SELECT visitorName, visitTime FROM
islands NATURAL JOIN visits
WHERE islandName = "Snowy Shores"
AND
YEAR(visitTime) = 2026 AND MONTH(visitTime) = 10
ORDER BY visitTime DESC
LIMIT 5;
```

3.4 Query 4

MongoDB Query

```php
// Query 4 - All islands that are in the southern hemisphere or have no native fruit recorded

echo "Query 4: \n";

$islandSouthOrNoFruit =
    $islands->find
    (
        [
            '$or' =>
                [['hemisphere' => 'south'],
                ['nativeFruit' => ['$exists' => false]]]
        ]
    );

foreach ($islandSouthOrNoFruit as $island)
{
    echo $island->islandName . "\n";
    echo $island->hemisphere . "\n";
    echo $island->nativeFruit . "\n";
    echo "\n";
}
```

Result

```
Query 4:
Lazy Atoll
south
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130


Golden Reef
south
Pancake

Tiny Island
north
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130


Misty Reef
south
Orange

Sunny Atoll
south
Strawberry

Tiny Atoll
north
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130


Lost Cay
south
Cherry

Cozy Key
south
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130


Snowy Shores
south
Pear

Lazy Haven
north
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130


Misty Atoll
south
Pear

Cozy Atoll
south
Pancake

Sunny Reef
north
PHP Warning:  Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130

Warning: Undefined array key "nativeFruit" in /var/www/html/a01/queries.php on line 130
```

```
Lost Isles
south
Pancake
```

SQL Equivalent

```sql
SELECT islandName, hemisphere, nativeFruit
FROM islands
WHERE hemisphere = 'south' OR nativeFruit = NULL;
```

4 Reports

4.1 Report 1

```php
// Per island: the number of visits in October 2026 and the time of the last one, busiest island first

echo "Per island: the number of visits in October 2026 and the time of the last one, busiest island first\n";
echo "\n";

$utc = new DateTimeZone('UTC');

$rep1 = $visits->aggregate
(
    [
        [
            '$match' =>
            [
                'visitTime' =>
                [
                    '$gte' => new UTCDateTime(new DateTimeImmutable('2026-10-01 00:00:00', $utc)),
                    '$lt'  => new UTCDateTime(new DateTimeImmutable('2026-11-01 00:00:00', $utc)),
                ],
            ],
        ],
        [
            '$group' =>
            [
                '_id'       => '$islandName',
                'visits'    => ['$sum' => 1],
                'lastVisit' => ['$max' => '$visitTime'],
            ],
        ],
        [
            '$sort' => ['visits' => -1]
        ],
        [
            '$project' =>
            [
                '_id'       => 0,
                'islandName' => '$_id',
                'visits'    => 1,
                'lastVisit' => 1,
            ],
        ],
    ]
);

foreach ($rep1 as $visit)
{
    echo $visit->islandName . " " . $visit->visits . " Last Visit: " . $visit->lastVisit->toDateTime()-
>format('Y-m-d H:i:s') . "\n";
}

echo "\n";
```

Output:

```
Per island: the number of visits in October 2026 and the time of the last one, busiest island first

Lazy Atoll 14 Last Visit: 2026-10-16 04:56:47
Wild Atoll 9 Last Visit: 2026-10-15 03:47:21
Lost Island 9 Last Visit: 2026-10-15 01:35:12
Misty Haven 9 Last Visit: 2026-10-16 14:53:15
Snowy Shores 9 Last Visit: 2026-10-16 09:17:20
Misty Atoll 8 Last Visit: 2026-10-16 06:38:51
Wild Key 8 Last Visit: 2026-10-14 08:10:44
Misty Island 8 Last Visit: 2026-10-16 08:52:12
Wild Shores 7 Last Visit: 2026-10-14 12:42:54
Cozy Island 7 Last Visit: 2026-10-16 02:00:20
Bizarre Isles 7 Last Visit: 2026-10-15 18:03:48
Lazy Key 7 Last Visit: 2026-10-15 07:47:12
Sunny Atoll 6 Last Visit: 2026-10-14 19:07:59
Cozy Shores 6 Last Visit: 2026-10-12 06:39:13
Sunny Cay 5 Last Visit: 2026-10-15 11:28:39
Lost Isles 5 Last Visit: 2026-10-15 21:09:28
Tiny Shores 5 Last Visit: 2026-10-14 08:01:20
Misty Cay 5 Last Visit: 2026-10-13 01:06:25
Misty Shores 5 Last Visit: 2026-10-15 07:08:48
Sunny Reef 5 Last Visit: 2026-10-13 23:45:39
Misty Reef 4 Last Visit: 2026-10-14 12:56:46
Lost Shores 4 Last Visit: 2026-10-15 06:34:09
Tiny Island 3 Last Visit: 2026-10-16 01:07:43
Lazy Haven 3 Last Visit: 2026-10-12 11:51:24
Cozy Atoll 3 Last Visit: 2026-10-16 11:15:12
Tiny Atoll 3 Last Visit: 2026-10-12 06:10:50
Tiny Key 2 Last Visit: 2026-10-10 11:30:09
```

SQL Equivalent

```sql
SELECT islandName, COUNT(visitTime) AS `Visits`, MAX(visitTime) AS `Last Visit`
FROM islands NATURAL JOIN visits
WHERE YEAR(visitTime) = 2026 AND MONTH(visitTime) = 10
GROUP BY islandName
ORDER BY `Visits` DESC;
```

4.2 Report 2

```php
// Visitors who came more than four times in total, most frequent first

echo "Visitors who came more than four times in total, most frequent first\n";
echo "\n";

$rep2 = $visits->aggregate
(
    [
        [
            '$group' =>
                [
                    '_id'       => ['$visitor', '$islandName'],
                    'visits'    => ['$sum' => 1],
                ],
        ],
        [
            '$match' =>
            [
                'visits' => ['$gte' => 5]
            ],
        ],
        [
            '$sort' => ['visits' => -1]
        ],
        [
            '$project' =>
                [
                    '_id'       => 0,
                    'visitEvent' => '$_id',
                    'visits'    => 1,
                ],
        ],
    ]
);

foreach ($rep2 as $visit)
{
    echo $visit->visitEvent[0] . " - " . $visit->visitEvent[1] . " - " . $visit->visits . "\n";
}

echo "\n";
```

Output:

```
Visitors who came more than four times in total, most frequent first

Blerg - Lost Island - 11
Kirsikka Lumikettu - Sunny Reef - 10
Mina Airfryer - Misty Cay - 10
Jack Storm - Lost Island - 10
Kirsikka Lumikettu - Lazy Atoll - 10
Kirsikka Lumikettu - Tiny Island - 9
Mina Airfryer - Golden Reef - 9
Kirsikka Lumikettu - Lost Island - 9
Kirsikka Lumikettu - Misty Reef - 9
Kirsikka Lumikettu - Cozy Shores - 9
Sasha Moi - Cozy Shores - 9
Kirsikka Lumikettu - Misty Haven - 9
Kirsikka Lumikettu - Wild Atoll - 9
Jade Arli - Lazy Key - 8
Kirsikka Lumikettu - Tiny Key - 8
Blerg - Sunny Atoll - 8
Kirsikka Lumikettu - Lazy Key - 8
Blerg - Misty Atoll - 8
Blerg - Misty Cay - 8
Blerg - Misty Shores - 8
Jade Arli - Golden Reef - 8
Jack Storm - Wild Atoll - 8
Sasha Moi - Lazy Atoll - 7
Blerg - Wild Atoll - 7
Mina Airfryer - Lazy Atoll - 7
Jade Arli - Tiny Island - 7
Jade Arli - Misty Haven - 7
Blerg - Lazy Key - 7
Blerg - Misty Haven - 7
Jack Storm - Lazy Key - 7
Mina Airfryer - Tiny Atoll - 6
Kirsikka Lumikettu - Misty Cay - 6
Blerg - Lazy Atoll - 6
Mina Airfryer - Tiny Key - 6
Mina Airfryer - Misty Atoll - 6
Jack Storm - Tiny Shores - 6
Jack Storm - Golden Reef - 6
Jade Arli - Wild Atoll - 6
Florence Meean'kar - Misty Haven - 6
Kirsikka Lumikettu - Misty Shores - 6
Kirsikka Lumikettu - Misty Atoll - 6
Florence Meean'kar - Lazy Atoll - 6
Jade Arli - Lazy Atoll - 6
Mina Airfryer - Misty Haven - 6
Jack Storm - Misty Haven - 6
Blerg - Tiny Shores - 6
Chris Roberts - Lost Island - 6
Mina Airfryer - Tiny Shores - 5
Mina Airfryer - Lazy Key - 5
Blerg - Golden Reef - 5
Kirsikka Lumikettu - Cozy Atoll - 5
Jack Storm - Lazy Haven - 5
Kirsikka Lumikettu - Lazy Haven - 5
Kirsikka Lumikettu - Lost Cay - 5
Jade Arli - Bizarre Isles - 5
Mina Airfryer - Snowy Shores - 5
Sasha Moi - Misty Atoll - 5
Blerg - Wild Key - 5
Blerg - Cozy Shores - 5
Kirsikka Lumikettu - Cozy Island - 5
Jade Arli - Misty Cay - 5
Kirsikka Lumikettu - Snowy Shores - 5
Blerg - Lost Cay - 5
Blerg - Lazy Haven - 5
Jade Arli - Tiny Key - 5
Mina Airfryer - Sunny Atoll - 5
Mina Airfryer - Lost Cay - 5
Blerg - Misty Island - 5
```

SQL Equivalent

```sql
SELECT visitorName, islandName, COUNT(visitTime) AS `Visits`
FROM islands NATURAL JOIN visits
GROUP BY visitorName, islandName
HAVING `Visits` > 4;
```

